

CosmoScreen

Szakmai dokumentáció

1. Összefoglaló:

A projektünk egy online mozis platform, amely átfogó és naprakész információkat nyújt a felhasználóknak. A weboldal célja, hogy átlátható és esztétikus formában mutassa be az aktuális filmkínálatot. A felület felhasználóbarát felépítésének köszönhetően a látogatók gyorsan és egyszerűen eligazodnak az oldalon, miközben magas szintű felhasználói élményben részesülnek. A platform célja, hogy központi információs felületként segítse a filmkedvelőket a tájékozódásban. Emellett hozzájárul a modern, digitális moziélmény erősítéséhez azáltal, hogy egy helyen, rendszerezett formában teszi elérhetővé a legfontosabb tudnivalókat.

2. Projekt bemutatása:

A projekt fejlesztése 2025. novemberében egy ambíciós ötletként indult. Melyet két személy vitt véghez. A fejlesztés 2026. áprilisában fejeződött be.

Fodor Adrián László: projektvezető

Kriván Balázs: projekt tag

2.1 Rendszerspecifikációk:

A rendszernek feladata, hogy ellássa az adminisztrációs feladatokat. Mégpedig az adatbázison végrehajtható CRUD műveleteket. Ebben a rendszerben csak adminisztrációs jogosultsággal lehet belépni.

2.2 Funkcionális követelmények

A rendszer az alábbi követelményeket tudja. Egy felhasználó barát bejelentkezési felület, ami csak az adminisztrációs engedéllyel rendelkező fiókokat engedi be. Kezdőlap/navbar az oldalak közötti navigálásra. Négy adatbázisból feltöltött információs oldal, amin végre tudjuk hajtani a CRUD műveletsorozatot.

2.3 Nem funkcionális követelmények

A rendszer egy modern stílust képvisel.

Reszponzív megjelenéssel rendelkezik.

Az alkalmazást egy telepítőn keresztül fel tudja telepíteni.

Az adatokat MySQL adattáblákban tároljuk.

3. A munka feltételei

3.1 Munkakörnyezet

A projekt a következő technológiákat/szoftvereket használja:

- MySQL adatbázis szerver, phpMyAdmin
- Programozási nyelv(ek): C#
- Git verziókövető
- IDE – Visual Studio 2022

3.2 Rizikómenedzsment

Betegség: Ez akkor lép fel amennyiben az egyik csapat tag lebetegszik.

Valószínűsége: közepes, hatása: gyenge

Megoldás: feladatok átcsoportosítása.

Szűk teljesítési idő: Mikor a projektre kitalált tervek megvalósítására idő szűkében vagyunk. Valószínűség: közepes, hatás: erős

Megoldás: Ütemképes feladat megoldás.

Konfliktusok: Amikor a elágazások ütköznek és konfliktust eredményez a kódban. Valószínűség: kicsi, hatás: közepes

Megoldás: A csapat tagok közötti kommunikáció.

4. Jelentések

4.1. Minőségbiztosítás

A minőségbiztosítás célja

A minőségbiztosítás célja annak biztosítása, hogy a fejlesztett mozi rendszer megbízhatóan működjön, és megfeleljen a meghatározott funkcionális és nem funkcionális követelményeknek. A folyamat segít a hibák korai felismerésében, valamint hozzájárul a stabil és felhasználóbarát működéshez.

Minőségbiztosítási folyamat

A projekt során a minőség biztosítása több lépésben történik:

- a fejlesztett funkciók rendszeres ellenőrzése
- a programkód felülvizsgálata és tesztelése
- a hibák dokumentálása és javítása
- az elkészült funkciók ismételt ellenőrzése

Tesztelési módszerek

A rendszer működésének ellenőrzése során különböző tesztelési módszereket alkalmazunk:

- **Funkcionális tesztelés:** annak ellenőrzése, hogy az egyes funkciók megfelelően működnek-e.
- **Manuális tesztelés:** a fejlesztő vagy tesztelő végig próbálja a rendszer funkcióit.
- **Hibakeresés:** a program működése során jelentkező hibák azonosítása és javítása.

Ellenőrzött funkciók

A mozi rendszer esetében a következő fontos funkciók kerülnek ellenőrzésre:

- filmek listázása és megjelenítése
- felhasználói adatok kezelése
- navigáció és felhasználói felület működése

Hibakezelés

A tesztelés során talált hibák rögzítésre kerülnek. A hibák javítása után újabb tesztelés történik annak érdekében, hogy ellenőrizzük a javítás sikerességét, valamint, hogy a módosítás nem okozott-e új problémát a rendszerben.

Összegzés

A minőségbiztosítási folyamat biztosítja, hogy a mozi rendszer stabilan működjön, és megfeleljen a felhasználói elvárásoknak. A rendszeres tesztelés és ellenőrzés hozzájárul a hibák csökkentéséhez és a program megbízhatóságának növeléséhez.

4.2. Státuszjelentés

A státuszjelentés célja

A státuszjelentés célja a projekt aktuális állapotának bemutatása, valamint annak nyomon követése, hogy a fejlesztés a tervezett ütemezés szerint halad-e. A jelentés segít áttekinteni az elkészült feladatokat, a folyamatban lévő munkákat és az esetleges problémákat.

Projekt aktuális állapota

A mozi rendszer fejlesztése a tervezett funkciók megvalósításával halad. A projekt főbb részei közé tartozik az adatkezelés, a felhasználói felület kialakítása, valamint a jegyfoglalási rendszer működésének biztosítása.

Elkészült funkciók

A fejlesztés során az alábbi funkciók kerültek megvalósításra:

- filmek listázása és megjelenítése
- vetítési időpontok kezelése
- alapvető felhasználói felület kialakítása
- adatbázis kapcsolat létrehozása
- adatkezelési felület kialakítása
- jegyfoglalás

Folyamatban lévő feladatok

Jelenleg az alábbi funkciók fejlesztése vagy tesztelése zajlik:

- jegyfoglalási rendszer fejlesztése
- rendszer tesztelése és hibajavítás
- Az oldal teljes színű fordítása

Felmerült problémák

A fejlesztés során kisebb technikai problémák merülhetnek fel, például adatkezelési vagy megjelenítési hibák. Ezek dokumentálásra kerülnek, majd javítás után ismételt tesztelés történik.

Következő lépések

A projekt következő szakaszában a fejlesztés célja a hiányzó funkciók befejezése, a rendszer teljes körű tesztelése, valamint a dokumentáció véglegesítése.

Összegzés

A státuszjelentés alapján a projekt a terveknek megfelelően halad. A fejlesztés fő funkciói elkészültek vagy fejlesztés alatt állnak, és a következő lépések a rendszer stabilizálására és véglegesítésére irányulnak.

5. Részletes feladatlista

A feladatok megszervezésére, a **Trello** projektmenedzsment szoftvert használjuk.

A teljes projektet a projektmenedzser koordinálja.

A kártyák reprezentálják az egyes feladatokat, melyeket listákba csoportosítunk:

- Teendők
- Folyamatban
- Kész

Minden egyes feladatnak (kártyának) rendelkeznie kell:

- felelős személy
- a feladat hovatartozása(WEB,C#)
- munkát elvégző személy(ek)

Munkamegosztás:

- Adatbázis: Fodor Adrián László és Kriván Balázs
- UI & Reszponzivitás: Fodor Adrián László és Kriván Balázs
- Frontend: Fodor Adrián László és Kriván Balázs
- Backend: Fodor Adrián László és Kriván Balázs
- Dokumentációk: Fodor Adrián László és Kriván Balázs
- Telepítő: Fodor Adrián László és Kriván Balázs

6. A munka tartalma

6.1. Átadandók és határidők (mérőföldkövek)

D – dokumentáció

P – prototípus

B – béta verzió

V - végtermék

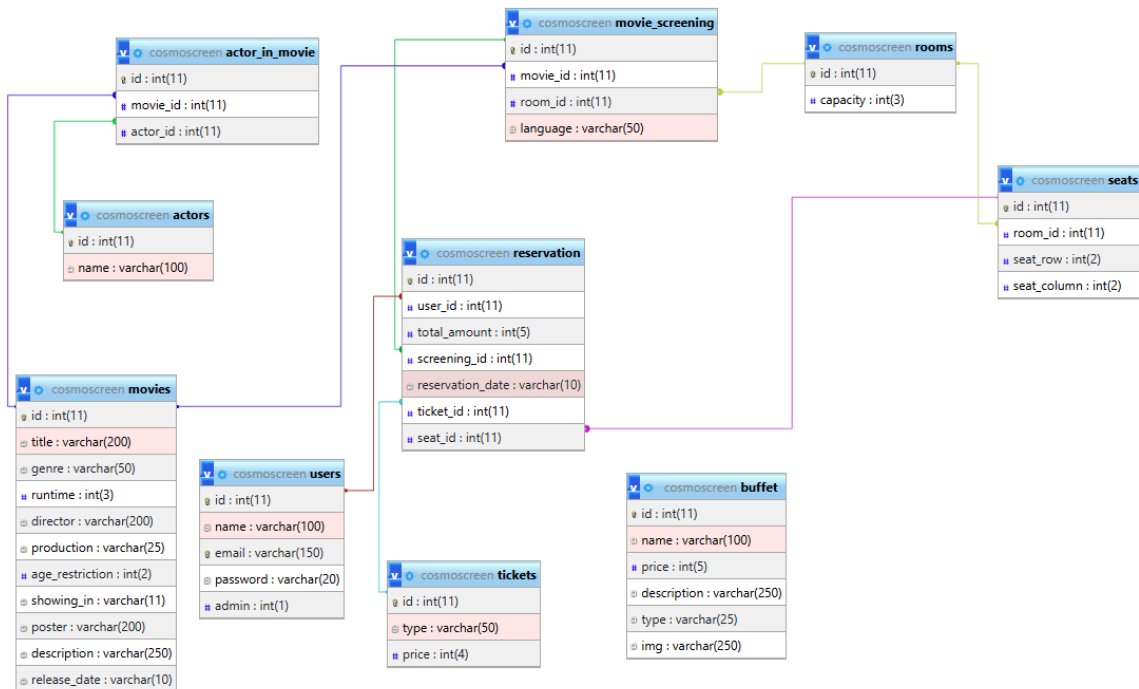
C#

Leadandó	Neve	Határideje
P1	Alap funkciók és kinézet	2025-11-27
P2	CRUD műveletek, 80% elkészültség	2025-12-21
P3+D1	Installer készítése és alap dokumentáció	2026-01-05
B1	Installer és 95%-os WPF	2026-02-06
D2	Felhasználói és szakmai dokumentáció	2026-03-13
D3	Teszt kódok és teszteredmények dokumentációja	2026-03-27
V+D4	A WPF alkalmazás véglegesítése, hibák kijavítása és a dokumentációk véglegesítése	2026-04-27

WEB

Leadandó	Neve	Határideje
P1	Szerkezeti felépítés	2025-11-10
P2	Alap oldalak létrehozása és alapvető funkciók	2025-12-10
P3	CRUD	2026-01-22
B1	Az oldal nagy mennyiségű funkcióinak működése, finomítás szükséges	2026-03-29
V+D1	A Weboldal és a dokumentációk véglegesítése	2026-05-05

7. Adatbázis struktúra



8. Forrás Kód

C#

Az alábbi metódusok a **methods.cs** fájlban találhatóak, ezekre azért volt szükségünk, hogy egy átláthatóbb kóddal tudjunk dolgozni, illetve, hogy elkerüljük a felesleges kódismétlést ezzel is tisztán tartva a programkódot.

```
16 references
public static void LoadData(string query, DataGridView datagrid)
{
    try
    {
        MySqlDataAdapter adapter = new MySqlDataAdapter(query, connection);
        openConnection();
        DataSet ds = new DataSet();
        adapter.Fill(ds);
        datagrid.ItemsSource = ds.Tables[0].DefaultView;
        closeConnection();
    }
    catch (Exception hiba)
    {
        MessageBox.Show(hiba.Message);
    }
}
```


LoadData metódus, ezt azért hoztuk létre, hogy egyszerűen tudjunk adatot betölteni és bármi változás esetén csak egy helyen kelljen módosítani a kódban.

Paraméterei: Kettő paramétert vár a metódusunk, az első egy **query, amit string formátumban kell megadni**. A második paraméterünk az egy **datagrid (táblázat)**, ami egyértelműen egy datagrid vagyis táblázat típusú paramétert vár. Ez a datagrid (táblázat) megjelenítésre szolgál.

Működési elve: Csatlakozik az adatbázishoz, a megadott query-t lefuttatja és a kapott adattal visszatér és a datagridbe azt feltölti.

```
1 reference
private void delete_data_btn_Click(object sender, RoutedEventArgs e)
{
    string torol = $"DELETE FROM movies WHERE id='{movie_id}'";
    movie_datagrid.SelectedItem = null;
    executeQuery(torol);
    LoadData("SELECT * FROM movies", movie_datagrid);
    delete_data_btn.IsEnabled = false;
    edit_data_btn.IsEnabled = false;
}
```

A **delete_data_btn_click** az egy eseménykezelő. Ez már nincs benne a methods.cs-ben. Itt egy törlés gomb működését láthatjuk. Amikor az Adminisztrátor valamit kijelöl és törlésre kattint akkor ezt az esemény hívódik meg.

Működési elve: Létrehozzuk a törléshez szükséges query-t, az adott elem id-ja segítségével, majd ezt a törlést végrehajtjuk és frissítjük az táblázatot. A módosít és a törlés gombot letiltjuk.

Fontos: Itt most nem térünk ki a CRUD többi három elemére külön-külön, mert lényegében ugyanazon az elven működnek, mint a törlés, csak más query-vel vagy gombokkal dolgoznak.

A **Logout** az egy metódus aminek a meghívása után megkérdezi a felhasználót hogy biztosan ki szeretne e lépni, ha igen akkor a megadott oldalt bezárja és a bejelentkezés oldalt mutatja.

Paraméterei: Egy paramétere van a metódusnak ami egy **Window** típusú **window** amikor meghívjuk a metódust akkor a bezárni kívánt oldalt adjuk meg.

Működési elve: Egy adott oldalon egy adott gombra meghívjuk a metódust amibe paraméterként egy Window-t adunk meg.

```
4 references
public static void Logout(Window window)
{
    MessageBoxResult result = MessageBox.Show(
        "Biztosan ki akar jelentkezni?",
        "Confirm",
        MessageBoxButton.YesNo,
        MessageBoxImage.Question);

    if (result == MessageBoxResult.Yes)
    {
        window.Close();
        ((App)Application.Current).ShowLogin();
    }
}
```

WEB

Az alábbi kód részletek a weboldalunk működési alapját és elvét mutatják be. Az itt látható kód a server.ts fájlból származik ahol a szerver felé történő kérések vannak.

Ez egy GET kérésre válaszoló útvonal amikor valaki meghívja akkor fut le a benne lévő lekérdezés ami jelen esetben az összes filmvetítés a filmek adataival együtt.

Ha sikeres akkor JSON formátumban visszaküldi az adatokat.

Ha sikertelen akkor azt ki írjuk konzolra és egy hibaüzenettel térünk vissza.

Ezen az alapon működik az összes szerver felé történő hívás, természetesen a belső lefutásra váró kód változó.

```
// get programs
app.get("/getPrograms", async(req,res) => {
  try {

    const response = await db.query(
      `SELECT movie_screening.id AS "screening_id",
         movie_id AS "movie_id",
         movies.title AS "movie_title",
         movies.poster,
         movies.runtime,
         movies.genre,
         movies.showing_in,
         room_id,
         language
      FROM movie_screening
      JOIN movies
      ON movie_id = movies.id`);

    res.status(200).json(response[0]);
  } catch (error) {

    console.log(error);
    res.status(500).json({ message: "Failed to get programs", error: true });
  }
})
```

Az itt látható kód a programListServices.ts fájlból származik mely az egyike a több service fájlnek amiknek célja hogy az adott oldalhoz tartozó kéréseket hajtsák végre.

A képen a getPrograms függvény látható mely meghívja a getPrograms végpontot mely az imént került bemutatásra. Visszaadja a szerver felől beérkező adatokat és hiba esetén null értékkel tér vissza.

A többi service fájl is ezen az alapon történik csak más végpontokkal és más visszatérési értékekkel.

```
// Get all movies method
async getPrograms() {
  try {
    const response = await axios.get(`${API_URL}/getPrograms`);
    return response.data;
  } catch (error) {}
  console.log('getPrograms failed:', error);
  return null;
},
```

Az itt látható kód a User.ts fájlból származik, amely egy a több típusdefiníciós (type) fájl közül. Ezeknek a fájloknak a célja, hogy átláthatóbbá tegyék a kódot azáltal, hogy központilag definiáljuk bennük az egyes objektumok szerkezetét, vagyis hogy milyen mezőkkel rendelkeznek és azok milyen típusúak.

A példában különböző interface-ek kerülnek definiálásra, amelyek eltérő felhasználói műveletekhez (például bejelentkezés, regisztráció, profilkezelés, jelszó módosítás) tartozó adatstruktúrákat írnak le.

Ezek az interfészek lehetővé teszik, hogy a frontendben típusosan és biztonságosabban kezeljük az adatokat.

A többi type fájl is hasonló elven működik: különböző funkciókhoz tartozó adatszerkezeteket definiálnak.

```
export interface LoginUserdata {
  email: string,
  password: string,
  showPassword: boolean,
}

export interface RegisterUserdata {
  name: string,
  email: string,
  password: string,
  showPassword: boolean,
}

export interface ProfilUserdata {
  name: string,
  email: string,
  isDisabled: boolean,
  originalName: string,
  originalEmail: string,
}

export interface PasswordUserdata {
  new_password: string,
  new_password_again: string,
  showPassword1: boolean,
  showPassword2: boolean,
}
```

Az itt látható kód az auth.ts-ből származik ami a user tárolását intézi a localStorageba. Ez egy state inicializáló függvény, amely a felhasználó adatait próbálja betölteni a böngésző localStorage-ából.

A függvény először kiolvassa a user kulcs alatt tárolt adatot. Amennyiben ez létezik és nem undefined az érték, akkor JSON formátumból JavaScript objektummá alakítja, és beállítja a state user mezőjébe. Ha nincs tárolt adat, akkor a user értéke null lesz.

Hiba esetén (például ha a tárolt adat nem érvényes JSON), a kód törli a hibás bejegyzést a localStorage-ból, majd a user értékét null-ra állítja.

```
//Data
state: () => {
  try {
    const stored = localStorage.getItem('user');
    return {
      user: stored && stored !== 'undefined' ? JSON.parse(stored) : null
    };
  } catch {
    localStorage.removeItem('user');
    return { user: null };
  }
},
```

9. Architektúra

A rendszer rétegzett architektúrát alkalmaz. A felhasználói felület (UI) kezeli az adatmegjelenítést, a háttérben futó logikai réteg végzi az adatfeldolgozást, míg az adatelérési réteg biztosítja a kapcsolatot a MySQL adatbázissal.

10. Irodalomjegyzék

W3School: <https://www.w3schools.com/>

ChatGPT: <https://openai.com/chatgpt/overview/>

C# dokumentáció: <https://learn.microsoft.com/en-us/dotnet/csharp/>

WPF dokumentáció: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>

imDb: <https://www.imdb.com/>

Stackoverflow: <http://stackoverflow.com/>